

React

Building UIs with Components

CSCI 39548 — Practical Web Development

Section Goals

- ▶ Understand components, JSX, and props
- ▶ Manage state with `useState` and side effects with `useEffect`
- ▶ Handle forms, lists, and data fetching
- ▶ Split components, extract custom hooks, and use refs
- ▶ Use React DevTools to inspect and debug

Section Items

- ▶ Setting Up with Vite
- ▶ JSX and Props
- ▶ useState
- ▶ Lists and Keys
- ▶ Forms
- ▶ Lifting State Up
- ▶ useEffect and localStorage
- ▶ Fetching Data
- ▶ Filters and Derived State
- ▶ Component Splitting
- ▶ Custom Hooks
- ▶ useRef and DevTools

Where Are We in the Course?

[Image: Course timeline: Git ✓, HTML/CSS ✓, JS ✓, TS ✓, then "React" highlighted and enlarged in the center, followed by Tailwind, Async Data, Routing, State, Backend still ahead]

Why React?

- ▶ The DOM is hard to keep in sync with your data
- ▶ React flips it: you **describe** what the screen should show. React **figures out** how.
- ▶ One pattern (components + state + props) scales from a toggle button to Facebook

What Is a Component?

```
function Hello() {  
  return <h1>Hello, world!</h1>;  
}
```

- ▶ A function that returns JSX
- ▶ That's the whole idea

Section Items

► **Setting Up with Vite**

- JSX and Props
- useState
- Lists and Keys
- Forms
- Lifting State Up
- useEffect and localStorage
- Fetching Data
- Filters and Derived State
- Component Splitting
- Custom Hooks
- useRef and DevTools

Setting Up with Vite

```
npm create vite@latest  
# Pick "React" -> "TypeScript"  
npm install  
npm run dev
```

- ▶ Scaffolds a new React + TypeScript project
- ▶ Hot reload — save and see changes instantly

The Files We Care About

- ▶ `index.html` — almost empty, just a `<div id="root">`
- ▶ `src/main.tsx` — entry point, hands the page to React
- ▶ `src/App.tsx` — your first component
- ▶ Everything else is config — ignore it for now

JSX Is JavaScript

- ▶ Looks like HTML, but it's JavaScript
- ▶ `<h1>Hello</h1>` compiles to `React.createElement("h1", null, "Hello")`
- ▶ Can't use `class` (reserved word) → use `className`
- ▶ Expressions go inside `{}`: `{name}`, `{2 + 2}`, `{items.length}`

- ▶ Open `src/App.tsx`, show the `<h1>`
- ▶ Change the text, save, watch the browser update instantly

Section Items

- Setting Up with Vite

► **JSX and Props**

- useState
- Lists and Keys
- Forms
- Lifting State Up

- useEffect and localStorage
- Fetching Data
- Filters and Derived State
- Component Splitting
- Custom Hooks
- useRef and DevTools

Components Take Props

```
function TodoCard({ todo }: { todo: Todo }) {  
  return <li>{todo.text}</li>;  
}
```

- ▶ Props = data flowing IN to a component
- ▶ Destructured from a single argument object
- ▶ TypeScript types catch wrong props at compile time

Passing Props

```
<TodoCard todo={todo1} />
```

- ▶ Like HTML attributes, but values are JavaScript inside `{}`
- ▶ Same Todo shape as in our TS demo: `{ id, text, done, minutes }`

One Component, Many Uses

- ▶ Same TodoCard renders three different todos
- ▶ That's the reuse win — write once, render anywhere
- ▶ Change the component → all instances update

- ▶ Add a 4th todo
- ▶ Render with `<TodoCard todo={todo4} />`